

COM06992-Estrutura de Dados-I
Período 2025-2
Trabalho 1

Professor: Dalvan Ribeiro de Almeida
Matrícula: _____
Nome: _____
Data de entrega: 23/02/2025

- Plágio não será tolerado. Trabalhos iguais serão penalizados com nota zero.
- O trabalho deverá ser feito em duplas.
- Você pode acrescentar as funções que julgar necessárias para completar as tarefas.
- Você deve criar as estruturas exatamente como especificado nesse trabalho e criar as funções com os mesmos protótipos dados (**IMPORTANTE**).
- Não deixe de indentar e documentar seu código para facilitar o entendimento do trabalho.
- Para entrega do trabalho, todos os arquivos devem possuir um cabeçalho com um comentário especificando o nome dos integrantes da dupla (como no exemplo abaixo). O aluno deverá compactar a pasta contendo todos os seus arquivos. Lembre-se de remover o arquivo executável da pasta antes de compactar. Apenas um membro da dupla deve enviar o trabalho pelo classroom, na atividade do trabalho.

```
1  /*Autores
2      Nome: Fulano de tal
3      Nome: Ciclano de tal
4  */
```

- O código enviado deve estar minimamente compilando sem erros. Erros de compilação serão penalizados com nota zero.

O tipo que representa um nó da árvore, para todo o trabalho, é dado por:

```
1  typedef struct arvore{
2      int  info;
3      struct arvore *esq;
4      struct arvore *dir;
5  }Arvore;
```

1. Considerando a implementação de árvores binárias apresentada em sala de aula, implemente as seguintes funções (recursivas):
 - (a) Uma função que retorne a quantidade de folhas de uma árvore binária. A função deve obedecer ao protótipo:

```
1 int qtdFolhas(Arvore* a);
```

- (b) Implemente uma função que, dada uma árvore, retorne a quantidade de nós que armazenam um inteiro c. A função deve obedecer ao protótipo:

```
1 int qtdInt(Arvore* a, int c);
```

- (c) Implemente uma função que compare se duas árvores binárias são iguais. Ela deve retornar 1 se as árvores forem iguais e 0, caso contrário. Essa função deve obedecer ao protótipo:

```
1 int iguais(Arvore* a, Arvore *b);
```

- (d) Uma função que, dada uma árvore, retorna uma cópia dessa árvore. Essa função deve obedecer ao protótipo:

```
1 Arvore* copia(Arvore *a);
```

- (e) Uma função que, dada uma árvore (de números inteiros positivos), retorne o maior elemento armazenado nela. A função deve retornar -1 se a árvore estiver vazia e deve obedecer ao protótipo:

```
1 int maior(Arvore *a);
```

2. Considere uma árvore binária de busca que armazena valores inteiros. Assim, os valores associados aos nós da sub-árvore à esquerda são menores que o valor associado à raiz e os valores dos nós da sub-árvore à direita são maiores.

- (a) Escreva uma função que retorne o número de ocorrências de um dado valor x na árvore. A função deve tirar proveito da ordenação da árvore e obedecer o seguinte protótipo:

```
1 int ocorrenciasX (Arvore *a, int x);
```

- (b) Escreva uma função que imprima os valores associados às folhas da árvore em ordem não crescente (isto é, do maior para o menor). A função deve obedecer ao protótipo:

```
1 void imprimirDecrescente(Arvore* a);
```

3. Crie um arquivo *main.c* e adicione uma função *main()* que teste o funcionamento de todas as funções do trabalho.